WEEK 12

Application Types & Architectures

Web Applications Development

Cyber.SoHo Educational Hub

"From the Industry to the Classroom — Building the IT's Future, Today and Together!"

🎯 Learning Objectives

By the end of today's class you will be able to:

- Sketch the 3 components of any web application
- Compare 1-tier, 2-tier, 3-tier, and N-tier architectures
- Decide between monolithic and microservices designs
- Distinguish SPA vs MPA, and SSR vs CSR
- Identify the 5 mobile application types
- Justify which mobile architecture fits which problem

Today's Roadmap

LESSON 1

Web Application Architectures

- 1.1 What is a web app architecture
- 1.2 Tier-based architectures
- 1.3 Monolithic vs Microservices
- 1.4 SPA vs MPA, SSR vs CSR

LESSON 2

Mobile Applications

- 2.1 Why mobile is its own world
- 2.2 The five main app types
- 2.3 Native apps in detail
- 2.4 Cross-platform & Hybrid
- 2.5 Progressive Web Apps (PWAs)

LESSON 1

Web Application Architectures

1.1 What Is a Web App Architecture?

Definition (1/3)

- A blueprint describing how a web application's parts are arranged.
- Like the architecture of a house: rooms (kitchen, bath, bedrooms) and the pipes connecting them.
- Move a 'room' and the plumbing changes — same with web apps.

1.1 Why Web App Architecture Matters

Why It Matters (2/3)

- **Performance — fast or slow.**
- **Scalability — survives 10 users? 10 million?**
- **Maintainability — fix one bug without breaking 14 others.**
- **Cost — your servers, bandwidth, and your future salary.**

1.1 The 3 Universal Components

Architecture Map (3/3)

FRONT-END

CLIENT

Browser / UI

→

BACK-END

SERVER

Logic / API

→

STORAGE

DB

Where data lives

Request flows left → right. Response flows right → left.

🔑 **KEY TAKEAWAY**

Every web app — without exception — has a Client, a Server, and a Database. The architecture is HOW they are arranged.

1.2 Tier-Based Architectures

Tiers — separate layers of responsibility

Tier	What lives where	Example
1-Tier	UI + logic + data on one machine	Microsoft Excel
2-Tier	Client talks directly to a database	Old desktop apps
3-Tier	Client → App Server → Database	Most modern web apps
N-Tier	Many specialised layers	Netflix, Google, Uber

1.2 Why Move From 2-Tier to 3-Tier?

Security & Maintainability

- In 2-Tier, the client knows the database password — anyone can decompile and steal.
- In 3-Tier, the App Server holds credentials and exposes only a clean API.
- Bonus: swap the DB (SQLite → PostgreSQL → MongoDB) without touching the client.
- CDN = Content Delivery Network — global cluster serving static content from the nearest location.

1.2 Tier Architectures

Key Takeaway

🔑 **KEY TAKEAWAY**

3-tier is the modern default. Adding the App Server in the middle protects your data and unlocks independent scaling.

1.3 Monolithic vs Microservices

Two opposite philosophies (1/3)

MONOLITH

- **One codebase, one deployment**
- ✅ Simple at first
- ✅ Easy to test locally
- ❌ Painful as it grows
- ❌ One bug can crash everything
- ❌ Redeploy ALL to update ANY part

MICROSERVICES

- **Many small, independent services**
- ✅ Independent teams & deployments
- ✅ Language freedom (Python, Go, JS…)
- ✅ Fault isolation
- ❌ Network complexity
- ❌ Harder debugging (distributed tracing)

1.3 When to Pick Which?

Architectural decision matrix (2/3)

Situation	Pick
Small team, early-stage startup	Monolith
Tight deadline, single use case	Monolith
Large team with mature DevOps culture	Microservices
Need fault isolation between features	Microservices
Need different teams shipping at different speeds	Microservices

1.3 Monolith vs Microservices

Key Takeaway (3/3)

🔑 **KEY TAKEAWAY**

Start with a monolith. Move to microservices only when the PAIN of the monolith outweighs the COMPLEXITY of microservices.

1.4 The Vocabulary You Need

Six abbreviations (1/3)

- **SPA = Single-Page Application — page loads once, JS swaps content.**
- **MPA = Multi-Page Application — every click loads a new page.**
- **SSR = Server-Side Rendering — server returns ready-built HTML.**
- **CSR = Client-Side Rendering — server returns near-empty HTML, JS builds in the browser.**
- **PWA = Progressive Web App — web app behaving like a native one.**
- **JAMstack = JavaScript + APIs + Markup — pre-built static files served by a CDN.**

1.4 SPA vs MPA — Side by Side

Side-by-side comparison (2/3)

Feature	MPA / SSR	SPA / CSR
First page load	Fast	Slow (downloads JS bundle)
Subsequent navigation	Slow (full reload)	Very fast
SEO	Excellent	Tricky (special handling)
Server load	Higher	Lower
Examples	Wikipedia	Gmail, Trello, Discord web

1.4 SPA vs MPA, SSR vs CSR

Key Takeaway (3/3)

🔑 **KEY TAKEAWAY**

SPA = great UX after first load. MPA = great SEO and first paint. Modern frameworks (Next.js, Nuxt, SvelteKit) deliver both — Universal Rendering.

Lesson 1 — Video Recommendation

▶ **Recommended Channel**

ByteByteGo

https://www.youtube.com/@ByteByteGo

Search on the channel for any of these:

- Search: 'The Evolution of Web Application Architecture'
- Why this channel: created by Alex Xu (System Design Interview author)
- Education-focused, ad-friendly, clear visuals

⏱ **Length under 15 minutes — perfect mid-class break**

Copyright safety: we only LINK to public YouTube content under YouTube ToS. Do NOT download or redistribute. Always verify the URL with VirusTotal before showing in class.

LESSON 2

Mobile Applications

2.1 Mobile Is Its Own World

Constraints that change the game

- Limited battery — no heavy background loops.
- Touch-first, not mouse-first — bigger targets, gestures.
- Variable network — Wi-Fi, LTE (Long-Term Evolution / 4G), or nothing.
- Two huge ecosystems — Apple iOS and Google Android.
- Hardware sensors — camera, GPS (Global Positioning System), NFC (Near-Field Communication), fingerprint.

2.1 Mobile vs Web

Key Takeaway

🔑 **KEY TAKEAWAY**

Mobile is NOT 'a smaller web'. Battery, touch, and offline must be designed in from day one.

2.2 The Five Mobile App Types (1/2)

Native, Mobile Web, Hybrid

Type	Built with	Best for
Native iOS	Swift + SwiftUI / UIKit	Top performance, full hardware
Native Android	Kotlin + Jetpack Compose	Top performance, full hardware
Mobile Web	HTML / CSS / JavaScript	Quick reach, no install
Hybrid	Web inside a WebView (Cordova/Ionic)	Internal tools, prototypes

2.2 The Five Mobile App Types (2/2)

Cross-Platform and PWAs

Type	Built with	Best for
Cross-Platform	React Native (JS/TS) or Flutter (Dart)	Most modern startups
PWA	Web + Manifest + Service Worker	Lean, installable, offline web

2.2 Five Tools, Five Jobs

Key Takeaway

🔑 **KEY TAKEAWAY**

The 'right' choice depends on team size, budget, performance needs, and time-to-market — not on which technology is trendy this year.

2.3 Native Mobile Apps

iOS + Android + a lot of code (1/2)

PROS

- **Built for ONE OS using its preferred language**
- iOS → Swift → Apple App Store
- Android → Kotlin → Google Play
- ✅ Maximum performance
- ✅ Full hardware access
- ✅ Polished native UX components

CONS

- **The price of going native**
- ❌ TWO separate codebases
- ❌ TWO separate teams (or one knowing both)
- ❌ TWO release cycles
- ❌ Twice the cost, twice the QA
- ❌ Day-1 of new OS features only here

2.3 When to Pick Native

Key Takeaway (2/2)

🔑 **KEY TAKEAWAY**

Pick native when performance and platform polish matter more than time-to-market. Examples: WhatsApp, Instagram, mobile games, AR (Augmented Reality) apps.

2.4 Cross-Platform vs Hybrid

Two ways to share code (1/3)

HYBRID (Cordova / Ionic)

- **Web app inside a WebView**
- ~95% code reuse
- ❌ Slowest performance
- ✅ Best for prototypes & internal tools
- Examples: older LinkedIn

CROSS-PLATFORM (RN / Flutter)

- **ONE real codebase, near-native speed**
- ~85% code reuse
- ✅ Fast development
- ✅ Fast performance
- Examples: Discord, parts of Airbnb, Tesla

2.4 Decision Matrix

Speed × Reuse × Time-to-market (2/3)

Approach	Speed	Reuse	Best for
Hybrid	Slowest	~95%	Quick wins
Cross-Platform	Near-native	~85%	Modern startups
Native	Fastest	0%	Games, AR, performance-critical

2.4 Cross-Platform Wins for Most

Key Takeaway (3/3)

🔑 **KEY TAKEAWAY**

Cross-platform is the DEFAULT modern choice for mobile startups. Pick native only when the use case truly demands it.

2.5 What Is a PWA?

Web + Magic = Mobile App (1/3)

- PWA = Progressive Web App — a website that BEHAVES like a native app.
- Installable from the home screen — no App Store needed.
- Offline-capable thanks to a Service Worker.
- Push-notification-capable (with limits on iOS).
- Discoverable via normal web search.

2.5 The Two Magic Ingredients

Manifest + Service Worker (2/3)

WEB APP MANIFEST

- **A small JSON file**
- Tells the browser the app's name
- Lists icons (192px, 512px…)
- Sets splash-screen colours
- Defines start_url and display mode
- Standardised by the W3C

SERVICE WORKER

- **A background JavaScript file**
- Runs even when the page is closed
- Caches files for offline use
- Handles push notifications
- Intercepts network requests
- Cache-first strategy = instant loads

2.5 Real-World Success: Twitter Lite

Case Study (3/3)

- Built for low-bandwidth markets (parts of Africa, Asia, South America).
- Under 1 MB total — vs 200+ MB for the native Twitter app.
- Loads in seconds even on 2G.
- Works offline thanks to the Service Worker.
- Installable straight from the website — no App Store review.
- Same code reaches iOS and Android users.

2.5 PWAs

Key Takeaway

🔑 **KEY TAKEAWAY**

PWAs are the leanest possible mobile architecture. No App Store, no Swift, no Kotlin — just three small files turn a website into an installable mobile app.

Lesson 2 — Video Recommendation

▶ **Recommended Channel**

Fireship

https://www.youtube.com/@Fireship

Search on the channel for any of these:

- Search: 'PWA in 100 Seconds'
- Search: 'React Native vs Flutter'
- Search: 'Why Native vs Cross-Platform Matters'

⏱ **Each video 2-10 minutes — pick one or stitch two together (under 15 min total)**

Copyright safety: we only LINK to public YouTube content under YouTube ToS. Do NOT download or redistribute. Always verify the URL with VirusTotal before showing in class.

RECAP

What to take home

🧠 12 Concepts to Remember (1/2)

Web architecture side

- 1. Client / Server / Database — the universal trio.
- 2. Tier architecture — 1, 2, 3, N tiers.
- 3. Monolith vs Microservices — when to use each.
- 4. SPA vs MPA — page-load behaviour.
- 5. SSR vs CSR — where rendering happens.
- 6. JAMstack & Universal Rendering — modern hybrids.

🧠 12 Concepts to Remember (2/2)

Mobile side

- 7. Native (Swift / Kotlin) — top performance.
- 8. Mobile Web — quick reach, no install.
- 9. Hybrid (Cordova / Ionic) — web in a WebView.
- 10. Cross-Platform (React Native / Flutter) — one codebase.
- 11. PWA (Manifest + Service Worker) — installable web.
- 12. Trade-offs — speed × reuse × cost × time-to-market.

🧠 Final Key Takeaway

Wrap-up

🔑 **KEY TAKEAWAY**

Architecture is NEVER about the 'best' technology — it is about the TRADE-OFFS.
The senior developer is the one who picks the right trade-off for the right project.
Today you started becoming that senior developer.

📝 Homework — For Next Class

Apply what you learned

- Open three apps on your phone right now.
- For each one, GUESS: native, hybrid, cross-platform, or PWA?
- Justify your guess in 2–3 sentences.
- Bring your three answers to next class — we will discuss.
- Tip: check the size of the install file and the App-Store description.

⚖️ Disclaimers

- Intellectual Property — Cyber.SoHo Educational Hub owns this material. No reproduction, redistribution, or commercial reuse without written permission.
- Copyright & No Affiliation — No affiliation, sponsorship, or endorsement is implied with Apple, Google, Meta, Amazon, Microsoft, Mozilla, W3C, IETF, X/Twitter, WhatsApp, Discord, Netflix, Uber, Airbnb, Tesla, Carrefour, Zara, Starbucks, Cordova, Ionic, Flutter, React Native, Next.js, ByteByteGo, Fireship, VirusTotal, or any other entity. Trademarks belong to their respective owners.
- External Links — Always verify external URLs with VirusTotal (https://www.virustotal.com/gui/home/url) before clicking or sharing in class.
- Liability & Ethics — Provided for educational purposes only. Apply ethically: respect privacy, GDPR, accessibility standards, copyright, and platform ToS.

THANK YOU

Questions, comments, debates?

Cyber.SoHo Educational Hub

"From the Industry to the Classroom — Building the IT's Future, Today and Together!"